

Summary of GDB commands for x86-64 Systems

Command	Effect
---------	--------

Starting (from shell):

<code>gdb</code>	
<code>gdb <file></code>	Define executable file
<code>gdb -args <file> 1 2 3</code>	Define command-line arguments 1 2 3
<code>gdb -ex 'b main' <file></code>	Execute gdb command 'b main' in gdb
<code>gdb -batch -ex 'disas' <file></code>	Execute command 'disas' in gdb and quit

Running and stopping (from gdb):

<code>quit</code>	Exit gdb
<code>run</code>	Run program
<code>run 1 2 3</code>	Run program with command-line arguments 1 2 3
<code>kill</code>	Stop the program
<code>Ctrl-d</code>	Exit gdb
<code>Ctrl-c</code>	Halts the current gdn command (does not exit gdb)

Breakpoints

<code>break sum</code>	Set breakpoint at the entry to function sum
<code>break *0x80483c3</code>	Set breakpoint at address 0x80483c3
<code>delete 1</code>	Delete breakpoint 1
<code>disable 1</code>	Disable the breakpoint 1 (gdb numbers each breakpoint you create)
<code>enable 1</code>	Enable breakpoint 1
<code>delete</code>	Delete all breakpoints
<code>clear sum</code>	Clear any breakpoints at the entry to function sum
<code>watch \$rax</code>	Trigger a breakpoint when the value of %rax changes

Execution

<code>stepi</code>	Execute one instruction
<code>stepi 4</code>	Execute four instructions
<code>nexti</code>	Like stepi, but proceed through function calls without stopping
<code>step</code>	Execute one C statement
<code>continue</code>	Resume execution until the next breakpoint
<code>until 3</code>	Continue executing until program hits breakpoint 3
<code>finish</code>	Resume execution until current function returns
<code>call sum(1, 2)</code>	Call sum(1,2) and print return value

Useful information

<code>backtrace</code>	Print the current address and stack backtrace
<code>where</code>	Print the current address and stack backtrace
<code>info program</code>	Print current status of the program)
<code>info functions</code>	Print functions in program
<code>info stack</code>	Print backtrace of the stack)
<code>info frame</code>	Print information about the current stack frame
<code>info registers</code>	Print registers and their contents
<code>info breakpoints</code>	Print status of user-settable breakpoints
<code>display /FMT EXPR</code>	Print expression EXPR using format FMT every time GDB stops
<code>undisplay</code>	Turn off display mode
<code>help</code>	Get information about gdb

TUI Layout

<code>layout src</code>	Displays source (C) and command windows
<code>layout asm</code>	Display disassembly and command windows
<code>layout split</code>	Display source, disassembly and command windows
<code>layout regs</code>	Add register content display to layout
<code>focus cmd</code>	Make command window active
<code>focus asm</code>	Make disassembly window active for scrolling

Examining code

<code>disas</code>	Disassemble current function
<code>disas sum</code>	Disassemble function sum
<code>disas 0x80483b7</code>	Disassemble function around 0x80483b7
<code>disas 0x80483b7 0x80483c7</code>	Disassemble code within specified address range
<code>print /x \$rip</code>	Print program counter in hex
<code>print /d \$rip</code>	Print program counter in decimal
<code>print /t \$rip</code>	Print program counter in binary

Examining data

<code>print /d \$rax</code>	Print contents of %rax in decimal
<code>p /d \$rax</code>	Short form
<code>p /x \$rax</code>	Print contents of %rax in hex
<code>p /t \$rax</code>	Print contents of %rax in binary
<code>p /d (int)\$rax</code>	Print contents of %rax in decimal after sign-extending lower 32-bits.

You need this to print 32-bit, negative numbers stored in the lower 32 bits of %rax. For example, if the lower 32-bits of %rax store 0xffffffff, you will see

```
(gdb) print $rax
$1 = 4294967295
(gdb) print (int)$rax
$2 = -1
```

<code>p 0x100</code>	Print decimal representation of 0x100
<code>p/x 555</code>	Print hex representation of 555
<code>p/x (\$rsp+8)</code>	Print (contents of %rsp) + 8 in hex
<code>p *(int *) 0xbffff890</code>	Print integer at address 0xbffff890
<code>p *(int *) (\$rsp+8)</code>	Print integer at address %rsp + 8
<code>p (char *) 0xbffff890</code>	Examine a string stored at 0xbffff890
<code>p *(short[10]*) 0xbffff890</code>	Print 10 consecutive shorts stored at 0xbffff890
<code>x/w 0xbffff890</code>	Examine (4-byte) word starting at address 0xbffff890
<code>x/w \$rsp</code>	Examine (4-byte) word starting at address in \$rsp
<code>x/wd \$rsp</code>	Examine (4-byte) word starting at address in \$rsp. Print in decimal
<code>x/2w \$rsp</code>	Examine two (4-byte) words starting at address in \$rsp
<code>x/2wd \$rsp</code>	Examine two (4-byte) words starting at address in \$rsp. Print in decimal
<code>x/g \$rsp</code>	Examine (8-byte) word starting at address in \$rsp.
<code>x/gd \$rsp</code>	Examine (8-byte) word starting at address in \$rsp. Print in decimal
<code>x/a \$rsp</code>	Examine address in \$rsp. Print as offset from previous global symbol.
<code>x/s 0xbffff890</code>	Examine a string stored at 0xbffff890
<code>x/20b sum</code>	Examine first 20 opcode bytes of function sum
<code>x/10i sum</code>	Examine first 10 instructions of function sum

(Note: the format string for the `x' command has the general form
x/[NUM][SIZE][FORMAT] where

NUM = number of objects to display
SIZE = size of each object: b=byte, h=half-word 16b, w=word 32b, g=giant 64b...
FORMAT = how to display each object: d=decimal, x=hex, o=octal, a=address...

If you don't specify SIZE or FORMAT, either a default value, or the last value you specified in a previous `print' or `x' command is used.

)

Links

Official GDB documentation:

<https://sourceware.org/gdb/current/onlinedocs/gdb/>

Beej's Quick Guide to GDB

<http://beej.us/guide/bggdb/>